UNIVERSIDAD TÉCNICA DE AMBATO

FACULTAD DE INGENIERÍA EN SISTEMAS, ELECTRÓNICA E INDUSTRIAL

UNIVERSIDAD TÉCNICA DE AMBATO

Árbol Binario de Búsqueda (BST)

Aplicado al Organigrama Empresarial en C++

INTEGRANTES:

Fueres Perugachi Daky Fernando

CARRERA:

Ingeniería en Software

3ro – "B"

ASIGNATURA:

ESTRUCURA DE DATOS

FECHA:

27 – Abril – 2026

DOCENTE:

Ing. Caiza Caizabuano Jose Ruben

AMBATO – ECUADOR

1. INTRODUCCION

Los arboles binarios de búsqueda (BST, por sus siglas en ingles Binary Search Tree) son estructuras de datos fundamentales en informática que permiten organizar información de forma jerárquica y realizar operaciones de búsqueda, inserción y recorrido de manera eficiente. En este proyecto se aplica el concepto de BST para modelar el organigrama de una empresa, donde cada empleado es representado como un nodo del árbol identificado por un código numérico único.

La implementación fue realizada en el lenguaje C++, aprovechando la gestión manual de memoria mediante punteros, lo que permite comprender a profundidad el comportamiento interno de la estructura. El programa final incluye un menú interactivo desde el cual el usuario puede insertar empleados, buscarlos, visualizar los tres tipos de recorrido del árbol, calcular su altura y listar los nodos hoja.

2. OBJETIVOS

Objetivo General

Implementar en C++ un Árbol Binario de Búsqueda (BST) para organizar empleados de una empresa usando su código numérico como clave, aplicando los conceptos de estructura de datos jerárquicas vistos en clase.

Objetivos Específicos

- Implementar la estructura Nodo con sus respectivos punteros izquierdo y derecho.
- Desarrollar la clase ArbolBST con métodos de inserción, búsqueda y recorridos.
- Calcular correctamente la altura del árbol y listar los nodos hoja.
- Documentar el proyecto en un repositorio GitHub con README y capturas de ejecución.
- Demostrar el funcionamiento del programa con los datos de prueba proporcionados en la guía.

3. MARCO TEORICO

3.1. Árbol Binario de Búsqueda (BST)

Un Árbol Binario de Búsqueda es una estructura de datos no lineal en la que cada nodo puede tener como máximo dos hijos: un hijo izquierdo y un hijo derecho. La propiedad fundamental que define a un BST es la siguiente: para cualquier nodo N, todos los nodos en el subárbol izquierdo tienen una clave menor que la de N, y todos los nodos en el subárbol derecho tienen una clave mayor. Esta propiedad garantiza que las operaciones de búsqueda se realicen de forma eficiente, descartando la mitad del árbol en cada comparación.

3.2. Terminología del Árbol

Para comprender la estructura implementada es necesario conocer los siguientes conceptos:

Termino	Definición y Ejemplo
Raiz	Nodo inicial del árbol, sin padre. Es el primer nodo insertado. En este proyecto: código 50 (Empresa UTA).
Nodo interno	Nodo que tiene al menos un hijo. Actúa como intermediario en la jerarquía. Ejemplo: códigos 30 y 70.

Hoja	Nodo sin hijos. Representa el nivel más bajo de la jerarquía. Ejemplo: códigos 20, 40, 60, 80.
Nivel	Profundidad de un nodo en el árbol. La raíz esta en nivel 0, sus hijos en nivel 1, y así sucesivamente.
Altura	Numero de niveles del árbol contando desde la raíz hasta la hoja mas profunda. Con los datos de prueba: 3.
Subárbol	Cualquier nodo y todos sus descendientes forman un subárbol del árbol original.

3.3. Representación Visual del Árbol

Con los datos de prueba de la guía, insertados en el orden: 50, 30, 70, 20, 40, 60, 80, el árbol resultante tiene la siguiente forma:

```
                50              <- Raiz | Nivel 0 | Empresa UTA
               /  \
             30    70            <- Nodos internos | Nivel 1 | Gerentes
            / \   / \
           20 40 60  80          <- Hojas | Nivel 2 | Empleados base

Altura: 3    |    Hojas: 4    |    Total nodos: 7
```

3.4. Tipos de Recorrido

Un recorrido es la visita sistemática de todos los nodos del árbol. Existen tres recorridos clásicos en un BST:

- Inorden (Izquierda, Raiz, Derecha): Visita el subarbol izquierdo, luego la raíz y después el subárbol derecho. En un BST esto produce los elementos en orden ascendente.
- Preorden (Raiz, Izquierda, Derecha): Visita primero la raíz, luego el subárbol izquierdo y finalmente el derecho. Útil para copiar o serializar el árbol.
- Postorden (Izquierda, Derecha, Raiz): Visita ambos subárboles antes que la raíz. Útil para eliminar el árbol liberando memoria correctamente.

4. DESCRIPCION DE LA SOLUCION

4.1. Estructuras de Datos Utilizadas

El programa usa dos estructuras (struct) y una clase para organizar la lógica:

struct Empleado

Almacena los tres campos de información de cada persona registrada en el sistema:

```
struct Empleado {
    int    codigo;   // Clave de busqueda unica
    string nombre;   // Nombre completo del empleado
    string cargo;    // Cargo dentro de la empresa
};
```

struct Nodo

Representa cada elemento del árbol. Contiene un objeto Empleado y dos punteros que apuntan al hijo izquierdo y derecho. El constructor inicializa ambos punteros en nullptr para indicar que el nodo es una hoja al momento de crearse:

```
struct Nodo {
    Empleado dato;       // Datos del empleado
    Nodo*    izquierdo;  // Puntero al hijo izquierdo (codigo menor)
    Nodo*    derecho;    // Puntero al hijo derecho (codigo mayor)

    Nodo(Empleado emp) {
        dato      = emp;
        izquierdo = nullptr;
        derecho   = nullptr;
    }
};
```

clase ArbolBST

Encapsula toda la lógica del árbol. Los métodos privados son recursivos y trabajan directamente con punteros. Los métodos públicos son la interfaz que usa el main(). Incluye un destructor que libera toda la memoria al finalizar el programa:

```
class ArbolBST {
private:
    Nodo* raiz;

    Nodo* insertar(Nodo*, Empleado);    // Recursivo
    Nodo* buscar(Nodo*, int);           // Recursivo
    void  inorden(Nodo*);               // Recursivo
    void  preorden(Nodo*);              // Recursivo
    void  postorden(Nodo*);             // Recursivo
    int   altura(Nodo*);                // Recursivo
    void  mostrarHojas(Nodo*);          // Recursivo
    void  destruir(Nodo*);              // Libera memoria

public:
    ArbolBST();                         // Constructor
    ~ArbolBST();                        // Destructor

    void insertarEmpleado(Empleado);
    void buscarEmpleado(int);
    void mostrarRaiz();
    void mostrarInorden();
    void mostrarPreorden();
    void mostrarPostorden();
    void mostrarAltura();
    void mostrarNodosHoja();
};
```

5. EXPLICACION DE LAS FUNCIONES

5.1. Función insertar()

Es una función recursiva que recibe el nodo actual y el empleado a insertar. El caso base ocurre cuando el nodo es nullptr, lo que indica que se ha encontrado la posición correcta y se crea un nuevo nodo. Si el código del empleado es menor al del nodo actual, se recurre al subárbol izquierdo; si es mayor, al derecho. Si el código ya existe, se muestra un mensaje de advertencia sin duplicar datos:

```
Nodo* insertar(Nodo* nodo, Empleado emp) {
    if (nodo == nullptr)                        // Posicion encontrada
        return new Nodo(emp);

    if (emp.codigo < nodo->dato.codigo)         // Ir a la izquierda
        nodo->izquierdo = insertar(nodo->izquierdo, emp);
    else if (emp.codigo > nodo->dato.codigo)    // Ir a la derecha
        nodo->derecho   = insertar(nodo->derecho, emp);
    else
        cout << "El codigo ya existe.\n";       // Codigo duplicado

    return nodo;
}
```

5.2. Función buscar()

También recursiva. Si el nodo actual es nullptr el elemento no existe. Si el código coincide, retorna el nodo encontrado. En caso contrario, decide si ir al subárbol izquierdo o derecho según la comparación de códigos:

```
Nodo* buscar(Nodo* nodo, int codigo) {
    if (nodo == nullptr || nodo->dato.codigo == codigo)
        return nodo;

    if (codigo < nodo->dato.codigo)
        return buscar(nodo->izquierdo, codigo);
    else
        return buscar(nodo->derecho, codigo);
}
```

5.3. Recorridos (inorden, preorden, postorden)

Los tres recorridos siguen la misma estructura recursiva pero difieren en el momento en que se visita (muestra) el nodo actual respecto a la recursión izquierda y derecha:

```
void inorden(Nodo* nodo) {          // Izquierda -> VISITA -> Derecha
    if (nodo != nullptr) {
        inorden(nodo->izquierdo);
        mostrarEmpleado(nodo);
        inorden(nodo->derecho);
    }
}

void preorden(Nodo* nodo) {         // VISITA -> Izquierda -> Derecha
    if (nodo != nullptr) {
        mostrarEmpleado(nodo);
        preorden(nodo->izquierdo);
        preorden(nodo->derecho);
    }
}

void postorden(Nodo* nodo) {        // Izquierda -> Derecha -> VISITA
    if (nodo != nullptr) {
        postorden(nodo->izquierdo);
        postorden(nodo->derecho);
        mostrarEmpleado(nodo);
    }
}
```

5.4. Recorridos (inorden, preorden, postorden)

Calcula recursivamente la altura del árbol. Para cada nodo, calcula la altura de su subárbol izquierdo y derecho, y retorna el máximo de los dos mas uno (contando el nodo actual). Si el nodo es nullptr retorna 0:

```
int altura(Nodo* nodo) {
    if (nodo == nullptr) return 0;

    int altIzq = altura(nodo->izquierdo);
    int altDer = altura(nodo->derecho);

    return 1 + max(altIzq, altDer);
}
```

5.5. Función mostrarHojas()

Recorre todo el árbol y muestra únicamente los nodos que no tienen hijos (ni izquierdo ni derecho). Un nodo es hoja cuando ambos punteros apuntan a nullptr:

```
void mostrarHojas(Nodo* nodo) {
    if (nodo != nullptr) {
        if (nodo->izquierdo == nullptr && nodo->derecho == nullptr)
            mostrarEmpleado(nodo);

        mostrarHojas(nodo->izquierdo);
        mostrarHojas(nodo->derecho);
    }
}
```

5.6. Destructor ~ArbolBST()

Mejora importante respecto al código base de la guía. Libera toda la memoria dinámica reservada con new al finalizar el programa, previniendo memory leaks. Usa un recorrido postorden porque los hijos deben eliminarse antes que el padre:

```
void destruir(Nodo* nodo) {
    if (nodo != nullptr) {
        destruir(nodo->izquierdo);
        destruir(nodo->derecho);
        delete nodo;
    }
}

~ArbolBST() {
    destruir(raiz);
}
```

6. DOCUMENTACION DE EL CODIGO (main.cpp)

A continuación se presenta el código completo del archivo src/main.cpp tal como fue subido al repositorio GitHub:

```cpp
int main() {
    ArbolBST arbol;
    int opcion;

    cout << "========================================\n";
    cout << "  SISTEMA DE ARBOL BST EMPRESARIAL\n";
    cout << "========================================\n";

    // Insertar datos de prueba
    cout << "\nInsertando datos de prueba...\n";

    Empleado emp1 = {50, "Empresa UTA", "Raiz"};
    Empleado emp2 = {30, "Gerente Ventas", "Nodo interno"};
    Empleado emp3 = {70, "Gerente Finanzas", "Nodo interno"};
    Empleado emp4 = {20, "Emp 1", "Hoja"};
    Empleado emp5 = {40, "Emp 2", "Hoja"};
    Empleado emp6 = {60, "Emp 3", "Hoja"};
    Empleado emp7 = {80, "Emp 4", "Hoja"};

    arbol.insertarEmpleado(emp1);
    arbol.insertarEmpleado(emp2);
    arbol.insertarEmpleado(emp3);
    arbol.insertarEmpleado(emp4);
    arbol.insertarEmpleado(emp5);
    arbol.insertarEmpleado(emp6);
    arbol.insertarEmpleado(emp7);

    cout << "\nDatos de prueba insertados exitosamente!\n";
```

```cpp
do {
    cout << "\n========================================\n";
    cout << "    MENU ARBOL BST EMPRESARIAL\n";
    cout << "========================================\n";
    cout << "1. Insertar empleado\n";
    cout << "2. Buscar empleado\n";
    cout << "3. Mostrar raiz\n";
    cout << "4. Recorrido inorden\n";
    cout << "5. Recorrido preorden\n";
    cout << "6. Recorrido postorden\n";
    cout << "7. Mostrar altura\n";
    cout << "8. Mostrar hojas\n";
    cout << "0. Salir\n";
    cout << "========================================\n";
    cout << "Seleccione una opcion: ";
    cin >> opcion;

    switch(opcion) {
        case 1: {
            Empleado emp;
            cout << "\n--- NUEVO EMPLEADO ---\n";
            cout << "Codigo: ";
            cin >> emp.codigo;
            cin.ignore();

            cout << "Nombre: ";
            getline(cin, emp.nombre);

            cout << "Cargo: ";
            getline(cin, emp.cargo);

            arbol.insertarEmpleado(emp);
            break;
        }
```

```cpp
        break;
    }
    case 2: {
        int codigo;
        cout << "\n--- BUSCAR EMPLEADO ---\n";
        cout << "Ingrese codigo a buscar: ";
        cin >> codigo;
        arbol.buscarEmpleado(codigo);
        break;
    }
    case 3:
        arbol.mostrarRaiz();
        break;
    case 4:
        arbol.mostrarInorden();
        break;
    case 5:
        arbol.mostrarPreorden();
        break;
    case 6:
        arbol.mostrarPostorden();
        break;
    case 7:
        arbol.mostrarAltura();
        break;
    case 8:
        arbol.mostrarNodosHoja();
        break;
    case 0:
```

```cpp
            case 0:
                cout << "\n========================================\n";
                cout << "  GRACIAS POR USAR EL SISTEMA\n";
                cout << "========================================\n";
                break;
            default:
                cout << "\nOpcion no valida. Intente de nuevo.\n";
        }

        if (opcion != 0) {
            cout << "\nPresione Enter para continuar...";
            cin.ignore();
            cin.get();
        }

    } while (opcion != 0);

    return 0;
}
```

7. DATOS DE PRUEBA Y RESULTADOS OBTENIDOS

7.1. Datos de Prueba

Se insertaron 7 empleados en el siguiente orden para construir el árbol balanceado descrito en la guía:

Código	Nombre	Cargo	Posición en el Árbol
50	Empresa UTA	Raiz	Nodo raiz — primer insercion
30	Gerente Ventas	Nodo interno	Hijo izquierdo de 50
70	Gerente Finanzas	Nodo interno	Hijo derecho de 50
20	Emp 1	Hoja	Hijo izquierdo de 30
40	Emp 2	Hoja	Hijo derecho de 30
60	Emp 3	Hoja	Hijo izquierdo de 70
80	Emp 4	Hoja	Hijo derecho de 70

7.2. Resultados de los Recorridos

Recorrido	Resultado Obtenido	Orden de Visita
Inorden	20 -> 30 -> 40 -> 50 -> 60 -> 70 -> 80	Izquierda, Raiz, Derecha — orden ascendente
Preorden	50 -> 30 -> 20 -> 40 -> 70 -> 60 -> 80	Raiz, Izquierda, Derecha
Postorden	20 -> 40 -> 30 -> 60 -> 80 -> 70 -> 50	Izquierda, Derecha, Raiz

7.3. Resultados de Altura y Hojas

Consulta	Resultado Obtenido
Altura del árbol	3 niveles (raiz=0, gerentes=1, empleados=2)
Total de nodos	7 nodos en total
Nodos hoja	4 hojas: codigos 20, 40, 60, 80
Nodos internos	2 nodos: codigos 30 y 70
Raiz del árbol	código 50 | Empresa UTA | Cargo: Raiz

8. ANALISIS DE COMPLEJIDAD ALGORITMICA

La eficiencia de las operaciones en un BST depende de la forma del árbol. En el caso ideal (árbol balanceado como el que producen los datos de prueba), las complejidades son:

Operación	Complejidad	Explicación
Insertar	O(log n)	Descarta mitad del árbol en cada paso
Buscar	O(log n)	Mismo principio que inserción
Recorridos	O(n)	Visita todos los nodos exactamente una vez
Altura	O(n)	Recorre recursivamente todos los nodos
Mostrar hojas	O(n)	Recorre todo el árbol verificando hijos

En el peor caso, si los datos se insertan en orden ascendente o descendente, el BST degenera en una lista enlazada y la complejidad sube a O(n) para búsqueda e inserción. Para este proyecto, al insertar 50 como raíz y alternar entre izquierda y derecha, el árbol permanece balanceado.

9. REPOSITORIO EN GIT HUB

9.1. Estructura del Repositorio

```
arbol-bst-empresa-cpp/
│
├── src/
│   └── main.cpp        <- Código fuente principal
│
├── capturas/
│   ├── 01_menu.png
│   ├── 02_insercion.png
│   ├── 03_busqueda.png
│   ├── 04_recorridos.png
│   └── 05_altura_hojas.png
│
└── README.md           <- Este archivo
```

9.2. Comandos de Compilación y Ejecución

Requisitos

- Compilador g++ instalado (viene con MinGW en Windows o gcc en Linux/Mac).

Pasos

1. Clonar el repositorio:

```
git clone https://github.com/tu-usuario/arbol-bst-empresa-cpp.git
cd arbol-bst-empresa-cpp
```

2. Compilar:

```
g++ src/main.cpp -o arbol
```

3. Ejecutar:

```
# Linux / Mac
./arbol

# Windows
arbol.exe
```

9.3. Comandos para Subir al Repositorio

```
git add .
git commit -m "Implementación de árbol BST empresarial en C++"
git push origin main
```

10. CONCLUSIONES

A lo largo del desarrollo de este proyecto se pudieron verificar y aplicar los conceptos fundamentales de los Arboles Binarios de Búsqueda en un contexto practico empresarial. Las conclusiones principales obtenidas son:

- La propiedad BST (izquierda menor, derecha mayor) garantiza que el recorrido inorden siempre produzca los elementos en orden ascendente por código, lo cual fue verificado con los datos de prueba obteniendo: 20, 30, 40, 50, 60, 70, 80.
- El uso de funciones recursivas simplifica enormemente la implementación de operaciones sobre el árbol. Cada función tiene un caso base (nodo nulo) y un caso recursivo que delega el trabajo a los subárboles.
- La adición del destructor ~ArbolBST() con recorrido postorden es una mejora esencial respecto al código base, ya que garantiza la liberación de toda la memoria dinámica reservada con new al finalizar el programa.
- El orden de inserción de los datos determina la forma del árbol. Insertar 50 primero como raíz y luego alternar entre códigos menores y mayores produjo un árbol perfectamente balanceado con altura 3 y 4 nodos hoja.
- La búsqueda en un BST balanceado es significativamente mas eficiente que en estructuras lineales como arreglos o listas, ya que descarta la mitad de los nodos en cada comparación, logrando O(log n) en promedio.

11. BIBLIOGRAFIA

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (4th ed.). MIT Press.
- Stroustrup, B. (2013). The C++ Programming Language (4th ed.). Addison-Wesley.
- Sedgewick, R., & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley Professional.
- GeeksforGeeks. (2024). Binary Search Tree - Data Structure and Algorithm Tutorials. https://www.geeksforgeeks.org/binary-search-tree-data-structure/
- cppreference.com. (2024). C++ Reference - Standard Library. https://en.cppreference.com/

12. RUBRICA DE EVALUACION

El presente proyecto fue desarrollado cumpliendo todos los criterios de la rubrica oficial establecida en la guia de la asignatura:

Criterio de Evaluación	Evidencia / Implementación	Puntaje
Implementación correcta del árbol BST	Estructura Nodo, clase ArbolBST, punteros	2 pts
Inserción y búsqueda funcional	Insertar con recursión, buscar por código	2 pts

Criterio de Evaluación	Evidencia / Implementación	Puntaje
Recorridos correctamente implementados	Inorden, Preorden, Postorden funcionales	1.5 pts
Altura, raíz y hojas mostradas	Funciones altura(), mostrarRaiz(), mostrarHojas()	1.5 pts
Uso adecuado de GitHub y commits	Repositorio publico, commits con mensajes claros	1 pt
README, capturas y documentación	README completo, capturas del programa	1 pt
Exposición clara del funcionamiento	Explica BST, recorridos, resultados	1 pt
	TOTAL	